Assessment Brief - Programming Paradigms Assignment - Full Details

COM225 Programming Language Principles Programming Paradigms Assignment

Submission details

17 December 2026

Date for return with feedback
12 January 2027

Task Description

In this assignment, you will implement a solution to an Abstract Syntax Tree (AST) problem using three distinct programming paradigms:

1. **Object-Oriented Paradigm** using Java
2. **Functional Paradigm** using Haskell
3. **Procedural Paradigm** using **either** Python (purely procedural style) or C

The objective is to write idiomatic code within each language and paradigm. Following implementation, you will submit a comparative report evaluating how language features influence system structure, state, type safety, and extensibility.

Submission Components
A complete submission consists of a single .zip file containing:

1. **Object-Oriented Solution:** java/Expr.java
2. **Functional Solution:** haskell/Expr.hs
3. **Procedural Solution:** python/expr_procedural.py **OR** c/expr.h & c/expr.c
4. **Written Comparative Report:** report.pdf (Guidance: max 1,000 words.)
5. **AI Declaration Form:** ai_acknowledgement.pdf (If Generative AI was used in any capacity)

Technical Domain Specification: Abstract Expression Evaluator
You must construct an Abstract Syntax Tree (AST) structure for arithmetic expressions involving numbers, binary operations, and variables.

Core AST Operations
Each solution must support four core operations across all three implementations:

- **Construction:** Build an expression tree representing a given mathematical expression.
- **Evaluation (**eval **/** eval_expr**):** Evaluate the expression given an environment mapping variable identifiers (e.g., "x", "y") to integer values.

- **String Formatting (**toString **/** to_string **/** print_expr**):** Convert the expression tree back into a human-readable, fully-parenthesized string with exact spacing (e.g., "(x + (3 * 2))").
- **Simplification (**simplify **/** simplify_expr**):** Recursively apply basic algebraic simplification rules:
    - 0 + e → e and e + 0 → e
    - 1 × e → e and e × 1 → e
    - 0 × e → 0 and e × 0 → 0
- Constant folding (e.g., evaluating 2 + 3 → 5) is **NOT** required unless one of the operands is 0 or 1 as specified in the rules above. For example, 0 + (2 + 3) → 2 + 3 (Simplifies zero rule, but leaves 2 + 3 as 2 + 3 without folding to 5).

Note: Simplification must operate bottom-up recursively so that nested identities (e.g., (0 * x) + (1 * y)) simplify fully to y in a single call. Children must be simplified **before** evaluating the current node's algebraic identities.

Minimal Expression Types Required

- **Literal / Number:** Integer constant (e.g., 5)
- **Variable:** Named identifier string (e.g., "x")
- **Add:** Addition node containing left and right sub-expressions
- **Multiply:** Multiplication node containing left and right sub-expressions

Implementation Requirements & Starter Patterns

Part A: Object-Oriented Implementation (Java)

Constraints for Automated Testing:

- Must use the default unnamed package (do not add a package declaration).
- Source file must be named Expr.java.
- Must expose public interface Expr and classes Num, Var, Add, Mul implementing Expr.
- Constructor signatures must be exact: Num(int value), Var(String name), Add(Expr left, Expr right), Mul(Expr left, Expr right).

```
import java.util.Map; // Core Interface: You must implement this interface across all node classes

public interface Expr {
    int eval(Map<String, Integer> env);
    String toString();
    Expr simplify();
}

// TODO: Implement concrete classes Num, Var, Add, and Mul implementing Expr.

// Example Stub for Add:
public class Add implements Expr {
    // TODO: Define private final fields and constructor signature: Add(Expr left, Expr right)
    @Override
    public int eval(Map<String, Integer> env) {
        throw new UnsupportedOperationException("Not implemented");
    }

    @Override
    public String toString() {
        throw new UnsupportedOperationException("Not implemented");
    }

    @Override
    public Expr simplify() {
        throw new UnsupportedOperationException("Not implemented");
    }
}
```

Part B: Functional Implementation (Haskell)

Constraints for Automated Testing:

- File must be named Expr.hs and declare module Expr where.
- AST data type must be named Expr with constructors Num Int, Var String, Add Expr Expr, Mul Expr Expr.
- Your environment type must be type Env = [(String, Int)].

```haskell
module Expr where

data Expr
    = Num Int
    | Var String
    | Add Expr Expr
    | Mul Expr Expr
    deriving (Show, Eq)

-- Environment definition for variable lookups
type Env = [(String, Int)]

-- Function Stubs: Implement using pattern matching and recursion
eval :: Env -> Expr -> Int
eval = error "TODO: Implement eval"

toString :: Expr -> String
toString = error "TODO: Implement toString"

simplify :: Expr -> Expr
simplify = error "TODO: Implement simplify"
```

Part C: Procedural Implementation (Python OR C)

Option C1: Python Implementation

Constraints for Automated Testing:

- File must be named expr_procedural.py
- Class names Num, Var, Add, Mul taking exact initialization parameters matching Java/C.
- The classes in Python are acting strictly as **C-style struct / record containers** (without methods), and all logic must reside in top-level procedures.

```
from typing import Dict, Union

# --- DATA STRUCTURES ---
class Num:
   def __init__(self, value: int):
       self.value = value

# TODO: Define Var, Add, Mul

# Type aliases for annotations
Expr = Union[Num, 'Var', 'Add', 'Mul']
Env = Dict[str, int]

# --- PROCEDURES ---
def eval_expr(expr: Expr, env: Env) -> int:
   """Evaluates an expression tree given a variable mapping environment."""
   raise NotImplementedError("TODO: Implement eval_expr")

def to_string(expr: Expr) -> str:
   """Returns a fully-parenthesized string representation of the expression tree."""
   raise NotImplementedError("TODO: Implement to_string")

def simplify(expr: Expr) -> Expr:
   """Recursively simplifies expressions applying zero and identity rules."""
   raise NotImplementedError("TODO: Implement simplify")
```

Option C2: C Implementation

Constraints for Automated Testing:

- Must be written in standard C99
- Your header file must be named expr.h, implementation file named expr.c
- You must have a struct of type Expr, ExprKind (enum with EXPR_NUM, EXPR_VAR, EXPR_ADD, EXPR_MUL), VarBinding and Environment.
- You must have these exact signatures: Expr *make_num(int), Expr *make_var(const char *), Expr *make_add(Expr *, Expr *), Expr *make_mul(Expr *, Expr *).

expr.h:

```c
#ifndef EXPR_H
#define EXPR_H

// TODO: Define ExprKind enum (EXPR_NUM, EXPR_VAR, EXPR_ADD, EXPR_MUL)
typedef enum {
   /* ... */
} ExprKind;

typedef struct Expr Expr;

// TODO: Complete the Tagged Union structure definition
struct Expr {
   ExprKind kind;
   union {
       /* Define variant payload structs/fields */
   } data;
};

typedef struct {
   const char *name;
   int value;
} VarBinding;

typedef struct {
   VarBinding *bindings;
   int count;
} Environment;

// Function Prototypes: You must implement these in expr.c
Expr *make_num(int value);
Expr *make_var(const char *name);
Expr *make_add(Expr *left, Expr *right);
Expr *make_mul(Expr *left, Expr *right);
void free_expr(Expr *expr);

int eval_expr(const Expr *expr, const Environment *env);
char *to_string(const Expr *expr);
Expr *simplify_expr(const Expr *expr);

#endif
```

Operational Contracts & Error Handling

To ensure compatibility with automated test execution:

1. **Parenthesisation Rules:** All binary expressions must produce fully-parenthesized strings with spaces surrounding operators: "(x + (3 * 2))".
2. **Unbound Variable Handling:**
    - Java: Throw IllegalArgumentException.

- Haskell: Call error "Unbound variable: <name>".
- Python: Raise KeyError or ValueError.
- C: Print error message to stderr and call exit(EXIT_FAILURE).

Pre-Submission Sanity Checks
Before submitting, run the provided sanity script for each language:

- **Java:** javac Expr.java SanityCheck.java && java SanityCheck
- **Haskell:** runghc SanityCheck.hs
- **Python:** python3 sanity_check.py
- **C:** gcc -std=c99 expr.c sanity_check.c -o sanity && ./sanity

Submissions that fail sanity checks due to mismatched module, function, or class names will receive an automatic zero for execution testing.

Written Report Structure (Guideline: max. 1,000 words)

Your written report must compare the three implementations by explicitly answering the following three core principles of programming language design:

- **The Expression Problem & Extensibility**
  - Compare what happens when you extend your program in two different directions:
    - **Adding a new data variant:** (e.g., adding a Power / exponentiation node).
    - **Adding a new operation:** (e.g., adding a differentiate(var) function).
  - Explain which language/paradigm required editing existing code versus adding brand-new isolated code. Frame your explanation using concepts like the **Open/Closed Principle** and **Dynamic Dispatch vs. Pattern Matching**.
- **Type Systems & Error Prevention**
  - Discuss how the type systems of Haskell, Java, and Python handled missing cases or invalid expressions (e.g., evaluating an unbound variable or handling unknown AST node types).
  - Contrast compile-time safety (static typing) with runtime checks (dynamic typing).
- **Modularity, Control Flow, & State**
  - Contrast how code is structured across the three solutions: function composition vs. method call chains vs. top-down procedures.
  - Discuss the presence or absence of mutable state and how it impacted your implementation of the simplify step.

Top Tips

1. **Pace yourself!** The programming tasks are not huge, but they will take time. You can start the Java solution **now** - you already learnt what you need for that one last year.
2. Embrace the idioms. The goal of this assignment isn't just to get the code working, it's to write code that feels natural in each paradigm.
3. Design your AST first, before writing any functions. Before writing eval or simplify, make sure your core tree node representations (Number, Variable, Add, Multiply) are defined cleanly in each language.
4. Watch out for memory and state. Simplification requires creating a *new* tree (or modifying an existing one) based on conditional rules (0 + e → e). Pay close attention to how state and memory differ across languages.

5. Anchor your report around the "expression problem." Focus on how easy or hard it was to code (aspects of) the solution in each language rather than explaining the code itself.
6. Use identical test cases against all three solutions. To verify that all three implementations are functionally equivalent, build a shared set of test expressions and environments.

Marking Criteria

You will be assessed on this subset of the faculty's generic marking criteria (https://docs.google.com/spreadsheets/d/1YY5S3PFAHqht5bAKAzorZmJ-VsFv-baBWoMoO6B6zFU/edit?usp=sharing):

1. Practical Skills
2. Knowledge & Understanding
3. Communication
4. Critical & Creative Thinking

Specific details about how these criteria will be assessed in the context of this assignment are given in this document (https://docs.google.com/spreadsheets/d/1zz8K3DTvLJqJ1v-gSA35khAHjQvM_zZr9bElEixvolY/edit?usp=sharing).

Referencing and Academic Integrity

You do not need to provide references for material that was presented as part of the course content for this module.

If you draw upon material beyond the course content, you must provide appropriate references - please refer to the library's guidance (https://sheffield.ac.uk/library/study/research-skills/referencing) on how to do this.

The use of Generative AI is not prohibited in this module, but if you use it, please use it in a responsible way and ensure that it is not undermining your learning. You should read this guidance (https://sheffield.ac.uk/study-skills/digital/generative-ai/assessment) from the study skills team and **if you do use Generative AI in any way to complete this assignment**, you must submit a PDF version of a completed Acknowledge, Describe, Evidence document (https://docs.google.com/document/d/1UXBBKWMJ4oIcsXtigETU0L82I2DHFBZy3bFxGajVrbQ/edit) (copy the template, fill it in as appropriate, download the PDF, then used the named link for submission).

Sources of support

If you need help with the programming elements of your assignment, you should seek help in the lab classes in the first instance. If you are unable to resolve your problem there, staff contact details are available on Blackboard. Please make sure you check the FAQ (if available) in the assessment folder. Wider sources of support and information can be found via these links: study skills@sheffield (https://www.sheffield.ac.uk/study-skills), late submissions, extenuating circumstances, extensions (https://students.sheffield.ac.uk/assessment)]

This document is based on Reading University: Assignment Briefs (https://www.reading.ac.uk/cqsd/-/media/project/functions/cqsd/documents/ade/tandl-resources/prp-inclusive-and-accessible-assignment-briefs.pdf).